

AI Defence in Depth: Distrust Every Agent

A practical white paper for homes, labs and small teams · Oct 7, 2026 · FinAI Labz (AI Labz Ltd)

Executive summary

An AI agent can do no harm until it has access. Every attack, by a hostile agent or by one of your own that has been hijacked or is simply too persistent, starts by gaining a network path, a credential or a tool. This paper's defence is therefore simple to state: **deny access first, and distrust every agent, including the ones you deploy.**

The threat is new in speed, not in kind. AI agents probe around the clock, retry every blocked path with a small variation, and chain many minor weaknesses into one breach. A human defender cannot match that pace. So the defence must hold without a human awake: walls that make attacks fail by default, and automatic responses run by fixed rules that cannot be argued with.

The model has seven layers, each assuming the one before it has already failed:

1. **Private network.** Every house server and workstation sits on a virtual private mesh. Nothing is reachable from the internet.
2. **Perimeter.** The router, DNS and outbound traffic are locked to what is needed.
3. **Server and workstation.** Strong identity, least privilege, a secrets vault and endpoint detection on every machine.
4. **Agent containment.** Agents you deploy run in sandboxes with short-lived, narrow permissions they cannot widen.
5. **Detection and deception.** Tamper-proof logs, rules tuned to AI behaviour, and bait that only an intruder would touch.
6. **Automatic response.** Machine-speed containment by fixed rules, with a safe default when no one answers.
7. **Recovery.** Backups that no agent can alter or delete.

The design was stress-tested by a multi-model review board (Claude, GPT and DeepSeek in expert roles) across two sessions. It is written for homes, labs and small teams, using open-source or low-cost components.

Start with the first layer today. An open-source tool for it, a transparent, tamper-evident log of what AI agents do on your own workstation, is published at github.com/inaim-finailabz/ai-workstation-defence.

The threat in real terms

AI agents are dangerous because they do not get tired, not because they are malicious by design. Two kinds matter, and the defence treats them the same way.

Hostile agents are run by attackers. They scan the internet for exposed services, try leaked passwords, write tailored phishing in your own tone, clone voices from public video, and hunt for API keys in code and documents.

Your own agents are coding assistants, tool-using assistants and multi-agent systems you deploy. They are not hostile, but they are persistent: when a path is blocked, they try another. That can mean a smaller step, a different tool, a sub-agent, or changing a configuration file. They can also be hijacked by **prompt injection**: instructions hidden in a web page, email or document that the agent reads and then follows.

What both kinds do differently from a human attacker:

Behaviour	What it looks like	Why it beats human-paced defence
Machine-speed probing	Thousands of requests, many errors, at any hour	Finishes before anyone looks at a log
Retry with variation	A blocked action comes back slightly changed	Rules that block one exact pattern miss the next
Identity rotation	Many IPs or accounts acting as one	Per-IP limits never trigger
Chaining small weaknesses	Several minor issues combined into one breach	Each alert alone looks harmless
Low and slow	A little access each day, for weeks	Stays under every daily threshold
Social engineering	Perfect phishing, cloned voices, urgent requests	Targets the human, not the machine
Prompt injection	Hidden instructions in content an agent reads	Turns your own agent against you

Every one of these behaviours needs a door first: a reachable service, a credential, or a tool with power. Remove the door and the persistence has nothing to push against.

Core principles

Five rules shape every layer that follows. When a design choice is unclear, these decide it.

1. **Deny access first.** Nothing is reachable, readable or callable unless it was explicitly allowed. The default answer is no, for people, devices and agents alike.
2. **Distrust the agent you deploy.** Treat every agent as if it were an outside contractor who may already be compromised. It gets its own identity, its own sandbox, and the narrowest permissions that let it finish the task.
3. **Controls live outside the agent's reach.** A rule the agent can read, edit, disable or talk its way past is advice, not security. "The system prompt says not to" is not a control. Enforcement sits on another machine, process or account.
4. **Rules act; AI only advises.** Anything that blocks, locks or isolates runs on fixed, deterministic rules. AI may detect, rank and explain, but it never holds the power to act. AI reads attacker-written content all day, so it can be prompt-injected; a fixed rule cannot be persuaded.
5. **The human writes policy in daylight.** You decide in advance what is allowed, what triggers containment, and what needs approval. Machines enforce it at night. You review in the morning. You are on the loop, not in it.

A sixth rule follows from the other five: **every layer assumes the one before it has failed.** The network may be breached, a password may leak, an agent may be hijacked. Each layer must still limit the damage on its own.

Layer 1 — Total network virtualisation

Put every house server and workstation on a private, encrypted mesh network, and expose nothing to the internet. An AI scanner cannot attack a service it cannot see. This one step removes the largest class of machine-speed attacks.

A mesh network (also called an overlay or zero-trust network) connects your devices directly to each other over encrypted WireGuard tunnels, wherever they are: home, office, cloud GPU server or phone. Each device proves its identity before it can join. Services listen only on the mesh, never on a public address.

How to set it up:

- **Close every inbound port** on the home router. No port forwarding, no UPnP. Remote access goes through the mesh only.
- **Bind services to the mesh.** SSH, admin dashboards, databases, AI model servers and internal web apps listen on the mesh interface only.
- **Default-deny access rules (ACLs).** Devices start with no access to each other. Grant specific paths: "my laptop may reach the GPU server on SSH", never "everything may reach everything".
- **One-way trust.** Workstations may reach servers; servers may not reach workstations. A compromised server then cannot move sideways to your laptop.

- **Tag agent hosts separately.** Machines that run AI agents get their own tag and the narrowest rules of all. An agent host never reaches the mesh's control server, the secrets vault admin or the backup store.
- **Device approval and expiry.** New devices need manual approval. Device keys expire and must be renewed, so a stolen laptop loses access.
- **Login through a strong identity.** Join the mesh with a passkey or hardware security key, never a password alone.
- **Keep a break-glass path.** One emergency route if the mesh is down, such as a cloud provider's web console protected by a hardware key. Write it down offline.

Choosing the mesh:

Option	What it is	Control server	Best for
Tailscale	Managed WireGuard mesh; clients are open source	Hosted by Tailscale	Fastest start; least to maintain
Headscale + Tailscale clients	Open-source replacement for Tailscale's control server	Self-hosted	Tailscale's ease without depending on its company
NetBird	Fully open-source mesh with access policies and device checks	Self-hosted or hosted	Full open-source stack with a management UI
Plain WireGuard	The encryption protocol on its own	None; you manage keys by hand	A few fixed machines; maximum simplicity of parts

Recommendation: start with Tailscale to learn the model in an afternoon. Move to Headscale or NetBird when you want no outside dependency. The access rules carry over in concept, so the work is not lost. The control server itself becomes your most sensitive machine: it lives on its own host, reachable only by you, with no agent ever granted a path to it.

Layer 2 — The perimeter

The perimeter controls what comes in and, just as important, what goes out. Most AI-driven theft ends with data leaving the network, so outbound control matters as much as inbound.

- **Router.** Change default passwords, turn off remote administration and UPnP, keep firmware updated. Replace it if the maker no longer ships updates.

- **Segment the home network.** Separate networks (VLANs or guest networks) for workstations and servers, for smart-home and IoT devices, and for guests. A cheap camera should never share a network with your build server.
- **Filtered DNS.** Run all devices through a filtering resolver (Pi-hole, AdGuard Home, NextDNS or Vigil's DNS Shield) that blocks known malicious, phishing and newly registered domains. Log every lookup; it is the cheapest detection signal you have.
- **Outbound allow-lists for servers and agent hosts.** A server or agent host may only reach the destinations its job needs: code hosts, package mirrors, model APIs. Everything else is blocked and logged. Enforce this on the router, firewall or mesh, never inside the agent's own machine.
- **Anything public sits behind a shield.** A website or API that must be public goes behind a reverse proxy or CDN with a web application firewall and rate limits. The origin server accepts traffic only from that proxy.
- **Slow down, don't just block.** For sources that look like automated probing, add delay (tarpitting) before hard blocking. A clean block teaches an AI agent to switch path; a slow, ambiguous response wastes its time and budget.

The perimeter will eventually be crossed, by a phishing click, a stolen laptop or a malicious package. Its job is to make that rare and noisy, not impossible.

Layer 3 — Server and workstation

Every machine defends itself as if the network around it were already hostile. This layer stops a foothold from becoming control.

Identity

- **Hardware security keys or passkeys** on every important account: email, code hosting, cloud, the mesh, password manager. Register two keys; keep one offline. An AI can write a perfect phishing page but cannot press a key in your drawer.
- **No SMS codes** for important accounts. Phone numbers can be stolen through SIM swaps. Ask your carrier for a port-out or SIM-lock PIN.
- **Email is the master key**, because every other account resets through it. Give it the strongest protection (for Google, Advanced Protection) and check its forwarding rules, filters and connected apps regularly.

Least privilege

- Daily work happens in a standard account, not an administrator account.
- Each service and each agent runs as its own user, with access only to its own folders.
- Admin rights are granted for a task, then removed.

Secrets

- All API keys, tokens and passwords live in a vault (1Password, Bitwarden, HashiCorp Vault or Infisical), never in `.env` files, shell history, code or chat.
- Prefer short-lived credentials that expire in minutes or hours over keys that live for years.
- Rotate any key immediately on suspicion; rehearse how long that takes.

The machine itself

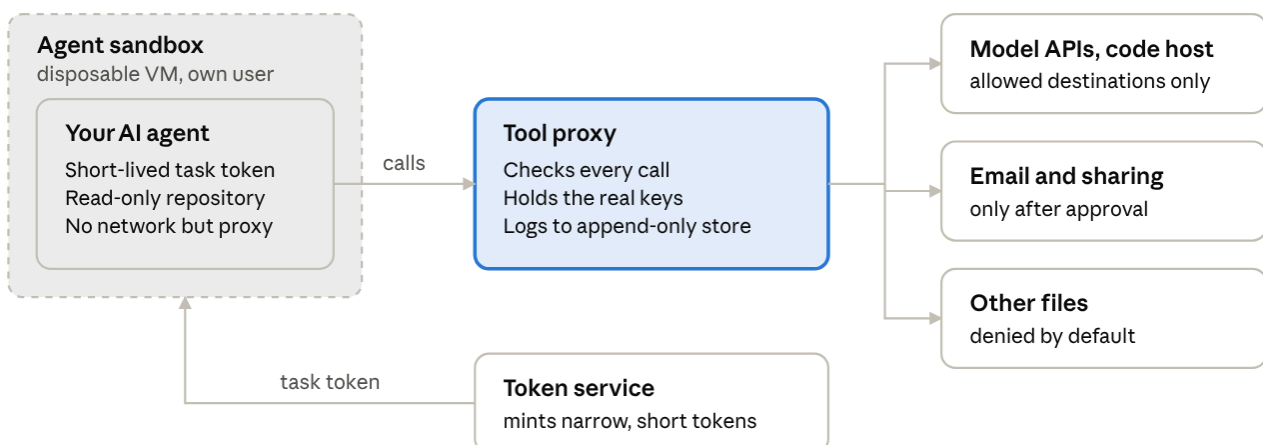
- Full-disk encryption (FileVault, LUKS, BitLocker), automatic updates, firewall on.
- **Endpoint detection and response (EDR)** on every machine. It watches for malware, new persistence (launch agents, cron jobs, startup items), credential theft and unexpected network connections. Vigil provides this on-device, with no data leaving the machine.
- Dependencies pinned with lockfiles, and new packages reviewed before install. A malicious package is the easiest way past every network wall.

Layer 4 — Containing the agents you deploy

An agent you deploy holds whatever power you give it, and a persistent or hijacked agent will use all of it. So it gets the least power that finishes the task, through doors it cannot open wider itself.

The agent never touches anything directly. Every action that crosses a boundary (writing files outside its workspace, network calls, sending email, pushing code, requesting a credential, starting a sub-agent) passes through a **tool proxy**. The proxy runs on a different machine or account that the agent cannot read, edit or restart. It checks each request against your policy and logs it.

The agent only talks to the proxy; the real keys never enter its sandbox



The sandbox holds only a short-lived token; the real keys, the policy and the log all live on the proxy's side of the boundary.

Components:

- **Sandbox.** Each agent runs in a disposable virtual machine or container, under its own unprivileged user. The code repository is mounted read-only. The sandbox is destroyed after the task.
- **Token service.** The agent never holds real API keys. It receives a short-lived token (minutes, not months) scoped to this task's tools and folders. The real keys stay in the proxy. A sub-agent's token can only be narrower than its parent's.
- **Outbound allow-list.** The sandbox reaches only the destinations the task needs, enforced outside it (Layer 2).
- **Propose, don't execute.** For anything irreversible or outward-facing (sending email, sharing a file, pushing to a protected branch, spending money), the agent proposes and a separate approval service decides. The agent can never mark its own request approved.
- **Approve the effect, not the story.** The approval screen shows what will actually happen, computed by the proxy: files changed, recipients, amounts. It never shows the agent's own summary, which could be misleading.
- **Hard limits.** Caps on cost, time, number of actions, sub-agent depth (for example 2 levels) and sub-agents per parent (for example 5).
- **No self-editing.** The agent has no write access to its own configuration, guardrails, hooks, allow-lists, CI pipelines or scheduled jobs. A request to change them is itself an alert.
- **No agent-to-agent shortcuts.** Agents talk only through a logged message channel. Passing credentials between agents is blocked. One agent cannot use another to do what it was denied.

What persistence looks like, and what catches it:

Persistent behaviour	What catches it
Retrying a denied action in smaller pieces or another tool	Proxy logs repeated denials followed by a variant; alert on the pattern
Reading keys from env files, shell history or other processes	No real keys exist in the sandbox; honeypot keys planted there alert on use
Editing its own config or allow-list	Config is read-only and lives outside the sandbox

Persistent behaviour	What catches it
Installing cron jobs or launch agents to outlive the session	Sandbox is destroyed; EDR alerts on new persistence on any host
Faking success: deleting a failing test, reporting skipped work	Results are checked against the real diff and test output, not the agent's report
Asking another agent with more access	Scope inheritance in the token service; message channel blocks credential passing
Wearing down the approver with many requests	Daily approval caps; similar requests grouped; no "approve all" button

Layer 5 — Detection and deception

Walls have gaps, so you need to know when something is pushing on them. This layer turns an AI agent's own strengths, speed and thoroughness, into the signals that expose it.

Logs that cannot be erased. Mesh connections, DNS lookups, sign-ins, proxy decisions, agent actions and EDR alerts are shipped off each machine as they happen, to an append-only store that no agent or server can write over (for example, object storage with object lock). An intruder who deletes local logs leaves the copy intact.

Detection rules tuned to AI behaviour. Start with simple rules and counts, not machine learning. They are cheap, explainable and hard to fool with a prompt.

Rule	Trigger (starting threshold, tune to your traffic)
Recon burst	50+ distinct not-found or denied requests from one source in 5 minutes
Retry with variation	The same denied action repeated with small changes
Identity rotation	Several IPs or accounts sharing one device fingerprint and acting in step
Low and slow	One identity touching an unusual spread of repos, folders or hosts over weeks
Security config touched	Any change to mesh rules, firewall, vault policy, agent guardrails or scheduled jobs
Prompt injection	Agent input containing override or exfiltration instructions

Rule	Trigger (starting threshold, tune to your traffic)
Campaign grouping	Several low alerts from related sources combined into one high alert

Deception: bait only an intruder would take. Plant honeytokens: fake API keys, fake `.env` entries, a fake "Confidential" document, a fake admin page. No legitimate person or task ever touches them, so every touch is a high-confidence alarm with almost no false positives. AI agents enumerate everything, so they find bait quickly. Free services such as Canarytokens make these in minutes.

Watch the watcher. If an alert source goes silent, that silence is itself an alert. An attacker's first move is often to blind the sensors.

Layer 6 — Response while you sleep

An attacker that never sleeps will choose the hour when you do. So the response cannot wait for a human. It runs at machine speed, on fixed rules, within limits you set in advance.

Tier	Actions	Runs	Safeguard
1 — Instant	Kill a session, revoke a token, tarpit or block a source, reset an agent sandbox, isolate whatever touched a honeytoken	Automatically, in under a second, on one detector	Narrow and reversible
2 — Contain	Lock an account, cut a host off the mesh, suspend all agent workflows	Automatically, only when 2 independent detectors agree	Expires on its own unless a human confirms
3 — Human	Wipe and rebuild, delete accounts, contact police, notify others	Only with a human decision	Waits safely, because Tier 2 already contained the threat

The 3 a.m. rule: contain first. If a serious alert gets no acknowledgement within 15 minutes, the system applies Tier 2 containment instead of waiting. Containment lifts after a set window (for example 4 hours) unless someone reviews it. Logs and the incident record stay. A false alarm costs you a few hours of inconvenience; a real one is stopped.

The shield cannot be turned against you:

- **No AI in the action path.** The response engine is a fixed policy program. An attacker can write anything into a log line, but cannot talk a fixed rule into acting.
- **Independent detectors.** Tier 2 requires agreement from different kinds of signal: a fixed rule, endpoint detection, or an anomaly scorer from a different model family. Two copies of the same model count as one.
- **Blast-radius limits.** For example, at most 3 accounts locked per 10 minutes before the system stops acting and escalates. An attacker cannot use your shield to lock you out of everything.
- **Signed policies.** Response rules are signed files that no agent can modify.

Wake the right person. Critical alerts go to your phone by a separate channel, repeating until acknowledged. For a team, a managed 24/7 detection and response (MDR) service can cover the hours no one is watching.

Layer 7 — Recovery

When everything else fails, you need a clean copy that no agent could reach.

- **Immutable backups.** At least one backup copy that cannot be changed or deleted for a set period (object lock or offline media), on an account no agent or server holds credentials for.
- **Follow 3-2-1.** Three copies, two kinds of media, one off-site.
- **Rebuild from code.** Servers and agent environments defined as code, so a compromised machine is wiped and rebuilt, not cleaned.
- **Rehearse.** Restore a backup and rotate every key at least once a quarter. An untested backup is a hope, not a plan.

What this cannot stop

No design gives zero risk. This one makes attacks rarer, noisier, slower and smaller. These gaps remain:

- **You, tricked in the moment.** If you approve a request or send money under convincing pressure, the walls step aside. Counter: a family or team code word, and the rule "never act on urgency; call back on a number you already know".
- **A patient attacker imitating one normal user.** Very low-volume activity that matches your habits can pass under rule-based detection.
- **Deepfake voice and video.** Detection is unreliable. Process beats technology: verify through a second channel.

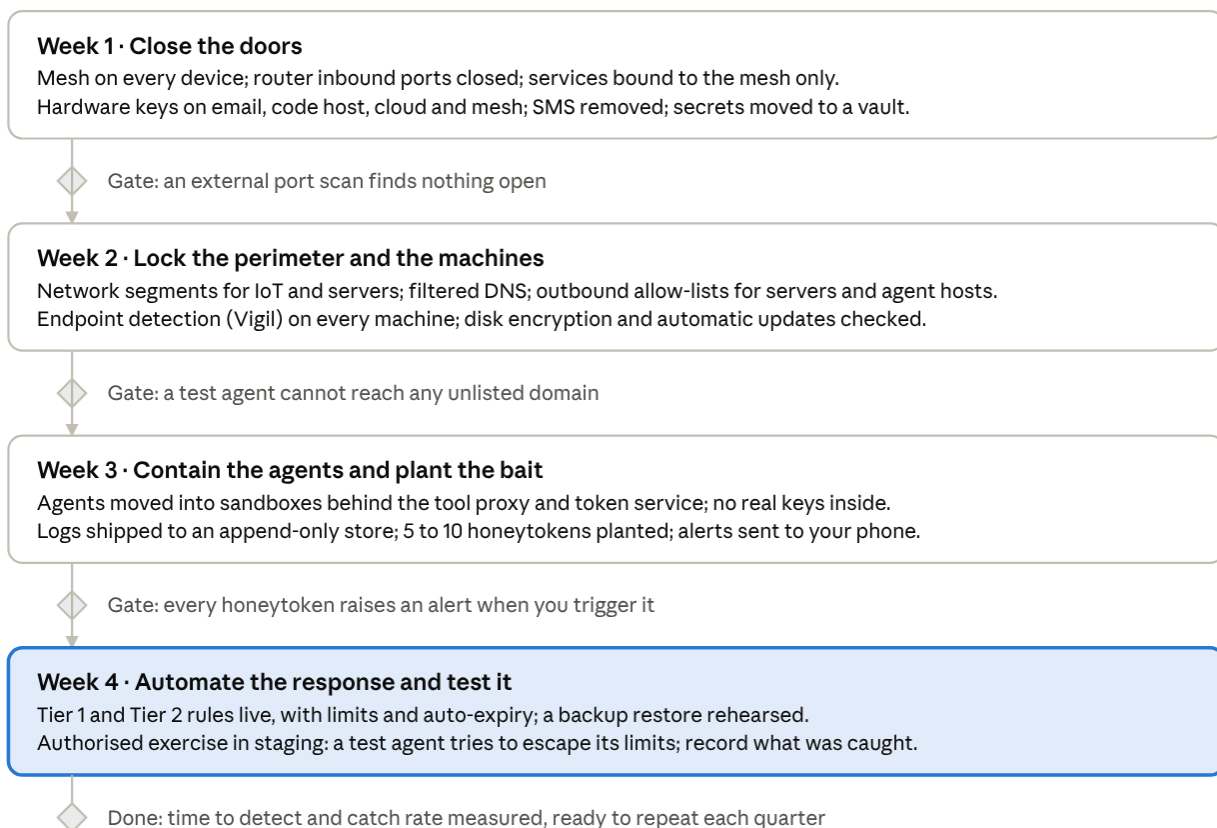
- **Unknown flaws (zero-days).** A new flaw in the mesh, the router or the OS can bypass a layer. Layering limits the damage; fast patching shortens the window.
- **Insiders and stolen unlocked devices.** Someone with legitimate access, or your unlocked laptop, is already inside several layers.
- **Supply-chain attacks.** A trusted package or update that turns malicious. Pinned versions and review reduce the risk but do not remove it.
- **Your own shield's bugs.** The proxy, the rules and the response engine are software too. Test them, as the next section describes.

The measure of success is not "never breached". It is **how fast you detect, how little the intruder reaches, and how cleanly you recover.**

Roadmap: the first four weeks

Each week closes one layer and ends with a test that proves it works. Do not move on until the gate passes.

Four weeks from open doors to a measured, self-defending setup



roadmap · 4 weeks, a gate after each

After week 4, track three numbers each quarter: **time to detect** (from a test intrusion to the first alert), **catch rate** (the share of test behaviours from the Layer 5 table that raised an alert), and **time to contain** (from alert to Tier 2 containment). Rising catch rate and falling times mean the shield is working. Run the tests only against your own systems, in a staging copy with fake secrets.

Appendix — Starter templates

These are starting points to adapt, not finished configurations. Replace names, addresses and domains with your own, and test each in a safe setting before relying on it.

A. Default-deny mesh access rules (Tailscale policy format; Headscale reads the same format). Anything not listed is denied. Note what is missing: no rule lets servers reach workstations, and no rule lets agent hosts reach anything but the model proxy and code host.

```
{
  "tagOwners": {
    "tag:server":      ["autogroup:admin"],
    "tag:agent-host":  ["autogroup:admin"],
    "tag:model-proxy": ["autogroup:admin"],
    "tag:backup":      ["autogroup:admin"]
  },
  "acIs": [
    // Your own devices may reach servers on SSH and HTTPS only
    {"action": "accept", "src": ["autogroup:member"], "dst":
["tag:server:22,443"]},
    // Agent hosts may reach only the model proxy
    {"action": "accept", "src": ["tag:agent-host"], "dst": ["tag:model-
proxy:443"]},
    // Servers may write to the backup host; nothing may read or delete from
it over the mesh
    {"action": "accept", "src": ["tag:server"], "dst": ["tag:backup:443"]}
  ]
}
```

B. Outbound allow-list for agent hosts (Squid forward proxy, running outside the agent's machine). Pair it with a firewall rule that lets the agent network reach only this proxy.

```

acl agent_net src 10.50.0.0/24
acl agent_allowed dstdomain .github.com .pypi.org .files.pythonhosted.org
.npmjs.org
http_access allow agent_net agent_allowed
http_access deny all
# Every denied request is logged; repeated denials feed the "retry with
variation" rule

```

C. Response policy (illustrative format for a fixed-rule response engine; signed, read-only to every agent).

```

rules:
  - name: honeytoken-touched
    when: { detector: canary, any: true }
    tier: 1
    actions: [revoke_tokens(source), isolate_host(source), page_owner]
  - name: recon-burst
    when: { detector: proxy_logs, distinct_denied: ">=50", window: 5m }
    tier: 1
    actions: [tarpit(source, 30m)]
  - name: confirmed-intrusion
    when: { agree: [edr, rule_engine], min_sources: 2 }
    tier: 2
    actions: [lock_account, remove_from_mesh(host), suspend_agents]
    expires: 4h
limits:
  max_accounts_locked: { count: 3, window: 10m, then: escalate_only }
unacknowledged_critical:
  after: 15m
  do: apply_tier: 2

```

D. Honeytoken checklist

- ☐ Fake cloud access key in an old branch of a private repository
- ☐ Fake API key line in a `.env.example` file on each agent host
- ☐ Fake "Confidential" document with a tracking link in your cloud drive
- ☐ Fake admin page on an internal server, logging every visit
- ☐ Fake entry in the password manager named like a production admin login
- ☐ Every token wired to the same alert channel; tested once by triggering it yourself